



Merge Intervals (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity
- Similar Problems

Problem Statement

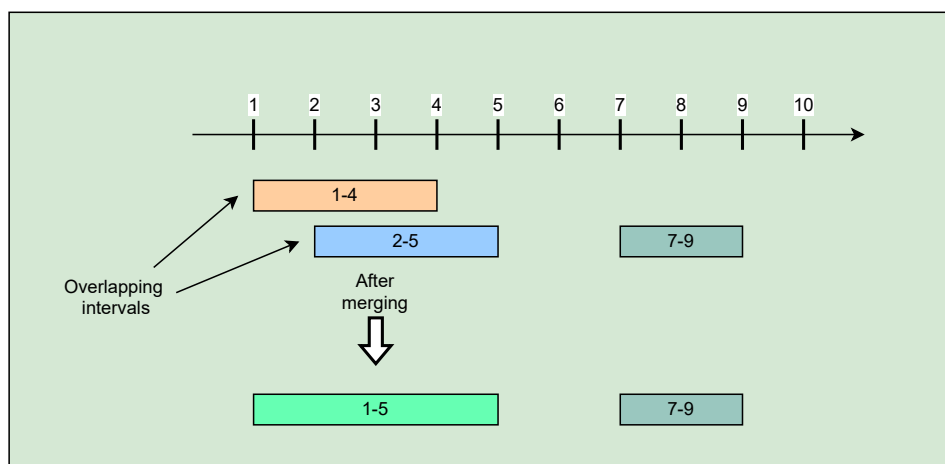
Given a list of intervals, **merge all the overlapping intervals** to produce a list that has only mutually exclusive intervals.

Example 1:

Intervals: `[[1,4], [2,5], [7,9]]`

Output: `[[1,5], [7,9]]`

Explanation: Since the first two intervals `[1,4]` and `[2,5]` overlap, we merged them into one `[1,5]`.



Example 2:

Intervals: `[[6,7], [2,4], [5,9]]`

Output: `[[2,4], [5,9]]`

Explanation: Since the intervals [6,7] and [5,9] overlap, we merged them into one [5,9].

Example 3:

Intervals: [[1,4], [2,6], [3,5]]

Output: [[1,6]]

Explanation: Since all the given intervals overlap, we merged them into one.

Try it yourself

Try solving this question here:



```
1 import java.util.*;
2
3 class Interval {
4     int start;
5     int end;
6
7     public Interval(int start, int end) {
8         this.start = start;
9         this.end = end;
10    }
11 };
12
13 class MergeIntervals {
14
15     public static List<Interval> merge(List<Interval> intervals) {
16         List<Interval> mergedIntervals = new LinkedList<Interval>();
17         // TODO: Write your code here
18         return mergedIntervals;
19     }
20
21     public static void main(String[] args) {
22         List<Interval> input = new ArrayList<Interval>();
23         input.add(new Interval(1, 4));
24         input.add(new Interval(2, 5));
25         input.add(new Interval(7, 9));
26         System.out.print("Merged intervals: ");
27         for (Interval interval : MergeIntervals.merge(input))
28             System.out.print "[" + interval.start + "," + interval.end + " ] ";
29         System.out.println();
30
31         input = new ArrayList<Interval>();
32         input.add(new Interval(6, 7));
33         input.add(new Interval(2, 4));
34         input.add(new Interval(5, 9));
35         System.out.print("Merged intervals: ");
36         for (Interval interval : MergeIntervals.merge(input))
37             System.out.print "[" + interval.start + "," + interval.end + " ] ";
38         System.out.println();
39
40         input = new ArrayList<Interval>();
41         input.add(new Interval(1, 4));
42         input.add(new Interval(2, 6));
43         input.add(new Interval(3, 5));
44         System.out.print("Merged intervals: ");
45         for (Interval interval : MergeIntervals.merge(input))
46             System.out.print "[" + interval.start + "," + interval.end + " ] ";
47         System.out.println();
```



```
48 }
49 }
```

Run

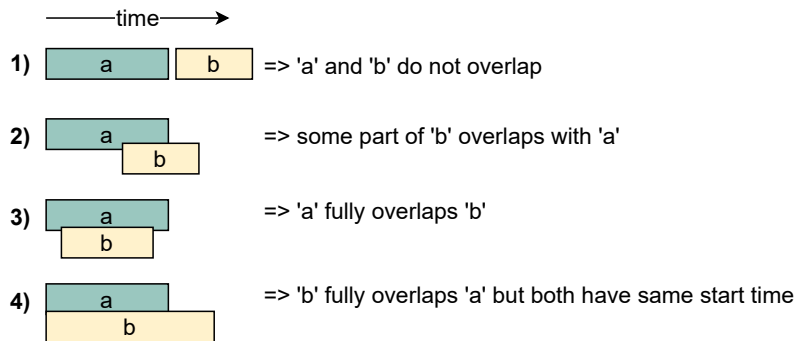
Save

Reset

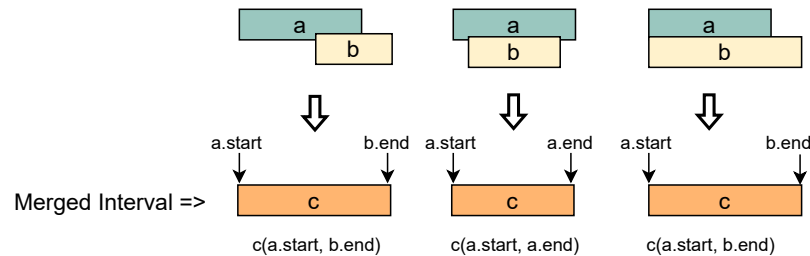


Solution

Let's take the example of two intervals ('a' and 'b') such that $a.start \leq b.start$. There are four possible scenarios:



Our goal is to merge the intervals whenever they overlap. For the above-mentioned three overlapping scenarios (2, 3, and 4), this is how we will merge them:



The diagram above clearly shows a merging approach. Our algorithm will look like this:

1. Sort the intervals on the start time to ensure $a.start \leq b.start$
2. If 'a' overlaps 'b' (i.e. $b.start \leq a.end$), we need to merge them into a new interval 'c' such that:

```
c.start = a.start
c.end = max(a.end, b.end)
```

3. We will keep repeating the above two steps to merge 'c' with the next interval if it overlaps with 'c'.

Code

Here is what our algorithm will look like:

Java

Python3

C++

JS

```
1 import java.util.*;
2
3 class Interval {
4     int start;
5     int end;
6
7     public Interval(int start, int end) {
8         this.start = start;
```



```
8     this.start = start;
9     this.end = end;
10 }
11 };
12
13 class MergeIntervals {
14
15     public static List<Interval> merge(List<Interval> intervals) {
16         if (intervals.size() < 2)
17             return intervals;
18
19         // sort the intervals by start time
20         Collections.sort(intervals, (a, b) -> Integer.compare(a.start, b.start));
21
22         List<Interval> mergedIntervals = new LinkedList<Interval>();
23         Iterator<Interval> intervalItr = intervals.iterator();
24         Interval interval = intervalItr.next();
25         int start = interval.start;
26         int end = interval.end;
27
28         while (intervalItr.hasNext()) {
29             interval = intervalItr.next();
30             if (interval.start <= end) { // overlapping intervals, adjust the 'end'
31                 end = Math.max(interval.end, end);
32             } else { // non-overlapping interval, add the previous interval and reset
33                 mergedIntervals.add(new Interval(start, end));
34                 start = interval.start;
35                 end = interval.end;
36             }
37         }
38         // add the last interval
39         mergedIntervals.add(new Interval(start, end));
40
41         return mergedIntervals;
42     }
43
44     public static void main(String[] args) {
45         List<Interval> input = new ArrayList<Interval>();
46         input.add(new Interval(1, 4));
47         input.add(new Interval(2, 5));
48         input.add(new Interval(7, 9));
49         System.out.print("Merged intervals: ");
50         for (Interval interval : MergeIntervals.merge(input))
51             System.out.print "[" + interval.start + "," + interval.end + " ] ";
52         System.out.println();
53
54         input = new ArrayList<Interval>();
55         input.add(new Interval(6, 7));
56         input.add(new Interval(2, 4));
57         input.add(new Interval(5, 9));
58         System.out.print("Merged intervals: ");
59         for (Interval interval : MergeIntervals.merge(input))
60             System.out.print "[" + interval.start + "," + interval.end + " ] ";
61         System.out.println();
62
63         input = new ArrayList<Interval>();
64         input.add(new Interval(1, 4));
65         input.add(new Interval(2, 6));
66         input.add(new Interval(3, 5));
67         System.out.print("Merged intervals: ");
68         for (Interval interval : MergeIntervals.merge(input))
69             System.out.print "[" + interval.start + "," + interval.end + " ] ";
70         System.out.println();
71     }
72 }
73
```

Run

Save

Reset



Time complexity



The time complexity of the above algorithm is $O(N * \log N)$, where 'N' is the total number of intervals. We are iterating the intervals only once which will take $O(N)$, in the beginning though, since we need to sort the intervals, our algorithm will take $O(N * \log N)$.

Space complexity

The space complexity of the above algorithm will be $O(N)$ as we need to return a list containing all the merged intervals. We will also need $O(N)$ space for sorting. For Java, depending on its version, `Collections.sort()` either uses [Merge sort](#) or [Timsort](#), and both these algorithms need $O(N)$ space. Overall, our algorithm has a space complexity of $O(N)$.

Similar Problems

Problem 1: Given a set of intervals, find out if any two intervals overlap.

Example:

```
Intervals: [[1,4], [2,5], [7,9]]
Output: true
Explanation: Intervals [1,4] and [2,5] overlap
```

Solution: We can follow the same approach as discussed above to find if any two intervals overlap.

[← Back](#)[✓ Completed](#)[Next →](#)[Introduction](#)[Insert Interval \(medium\)](#)